

User-Selectable Feed Algorithms for Community Posts

The gap your idea targets and why it matters now

Most large community platforms expose only *limited* controls over feed ranking, even when they offer multiple feed tabs. For example, X distinguishes a recommendation-driven “For you” timeline from a “Following” timeline that shows posts from followed accounts in reverse-chronological order, letting users switch between those modes but not truly define the ranking logic themselves. ¹

Similarly, Reddit’s Home feed has increasingly relied on machine-learning recommendations; Reddit explicitly describes “home feed recommendations” as ML-based recommendations intended to improve and personalize the “Best” sort, and it offers settings to disable those recommendations—but again, that’s a toggle, not an algorithm builder. ²

Meta’s products have gradually added alternatives (like “Following” / “Favorites” feeds on Instagram, and “Most recent”/recent sorting views on Facebook), but these options are still constrained to platform-defined modes and often aren’t set as the default experience. ³

What’s changed is that “algorithmic choice” has moved from theory into real, shipping ecosystems. Bluesky ⁴ is a key precedent: it introduced “Custom Feeds” where third-party “feed generators” can provide different ranking algorithms that users can choose and pin—explicitly positioning this as “algorithmic choice” instead of a single black-box ranking. ⁵ Tools like SkyFeed and Graze emerged to make that ecosystem usable (including “no-code” feed building). ⁶

The regulatory direction in the EU also supports your “user agency” framing: the EU Digital Services Act provisions on recommender systems push platforms toward clearer disclosure of main parameters and user-facing options to modify or influence them, and for the largest platforms, at least one recommender option not based on profiling. ⁷ Recent court developments around profiling-free chronological options for Meta services show that “feed choice” is increasingly scrutinized as a UI/UX *accessibility* issue, not just a backend feature. ⁸

Taken together, your idea is aligned with a real industry + research shift: moving from “one algorithm for everyone” toward “multiple algorithms per user,” with strong emphasis on usability and transparency. ⁹

The clean core concept: treat user algorithms as “Feed Recipes,” not code

The most important framing shift (for both security and UI comprehension) is to avoid the word “algorithm” in the **user-facing** product, even if your team uses it internally.

A workable “real-time product” interpretation is:

A Feed Recipe is a stored, named, user-owned configuration that selects and ranks posts using only pre-approved building blocks. ¹⁰

This matches what has proven viable elsewhere: - In Bluesky’s model, a feed generator returns a list of post URIs (a feed “skeleton”) that the client can hydrate into full posts; it’s a clean boundary that enables custom ranking without exposing arbitrary execution to users. ¹¹

- In no-code builders like Graze, feeds are described as buildable/deployable without writing code—i.e., configuration, templates, filters, sort order, and moderation settings rather than user-submitted programs. ¹²

So your product can be described (cleanly) as:

User-Owned Feed Recipes - Users *create or pick* a recipe (“My Trending”, “Fresh Updates”, “Only Friends”, “Deep Discussions”). - Each recipe controls: **sources** → **filters** → **ranking priorities** → **diversity rules** → **safety constraints** (all through UI, no code). - Users can store multiple recipes, choose one as default, and switch instantly without confusing the system. - If a recipe fails, the system falls back to your default algorithm and shows an understandable explanation.

This concept directly addresses your security constraint (“Don’t give code-based algorithms”) while still delivering genuine user agency. ¹³

What users can input without code: four safe “input channels” that can coexist

Your hardest problem is not ranking math; it’s collecting user intent in a way that: - non-technical users understand, - can be executed safely at scale, - can be debugged and supported when it fails.

Research on user control in recommender systems consistently finds a tradeoff: more expressiveness increases complexity and cognitive load, and users often hesitate to use powerful controls unless the UI makes outcomes obvious and low-effort. ¹⁴

A robust product uses **multiple input channels**, progressively disclosed.

Channel one: “Pick a recipe” (algorithm selection, not algorithm creation)

This is the simplest and most scalable form of “user-set algorithms”: let users choose between multiple ranking strategies that your platform supports.

This is supported by evidence that users do engage with the ability to switch recommenders, and that recommender systems are not one-size-fits-all. ¹⁵

Your UI could start with 6–10 human-readable recipe cards, for example: - “Fresh Updates” (recency-first) - “Trending Now” (velocity-first) - “Deep Discussions” (comment/reply activity-first) - “From People I

Follow” (chronological / near-chronological) - “Discover New Voices” (novelty + diversity) - “Low Noise” (filters out certain post types + repeated authors) - “Nearby / Local” (if your product has locality) - “My Interests” (topic-based)

Even if advanced creation exists, these presets are essential because research and real-world control features show most users won’t build from scratch unless the effort is very low and the benefit is immediate. ¹⁶

Channel two: “Knobs and sliders” (parameter control that feels intuitive)

There’s strong precedent for giving users control over a small number of meaningful variables (e.g., popularity vs. recency) because it’s: - easier to understand, - cheaper to compute, - easier to explain and debug. ¹⁷

A minimal but powerful set of knobs that map to “Trending” and “New updates first” could be:

- **Freshness ↔ Popularity** (time decay vs engagement)
- **Following ↔ Beyond my network** (source scope)
- **More variety ↔ More of what I like** (diversity vs similarity)
- **Fewer repeats** (cap one author / one topic per N posts)
- **Low-conflict ↔ Unfiltered** (content tone / sensitivity, if you have signals)

This matches both classic research on user-control mechanisms ¹⁸ and the “design objectives” identified in recent social feed customization research emphasizing understandability and integration into normal browsing. ¹⁹

Channel three: “Teach the feed by example” (interactive machine teaching)

This is the best answer to your requirement that *non-knowledgeable people* should still be able to “set their own algorithm.”

Modern HCI research frames this as **Interactive Machine Teaching (IMT)**: the user is the “teacher,” the system is the “learner,” and the user refines the feed by giving feedback on real items while browsing. ²⁰

Two highly relevant findings from recent work on teachable/customizable social media feeds: - Controls must be available when users want them, but not intrusive; customization should live alongside normal browsing. ²⁰

- There is a constant tension between expressiveness and understandability; the UI should minimize cognitive load while still letting users steer outcomes. ²¹

Concretely (as a product idea, not implementation): your feed can embed micro-controls like: - “Show me more like this” - “Show me less like this” - “I want fewer posts like this topic” - “I want more from this community” - “I don’t want posts like this at night”

This “teaching” channel becomes the ongoing way users maintain their recipe over time, instead of forcing them to design logic upfront. ²⁰

Channel four: “Natural language recipe builder with procedural transparency” (AI-assisted, but constrained)

Your AI idea is viable *if* you treat AI as a translator from intent → safe config (not as an autonomous ranking brain).

The latest research direction explicitly explores natural-language feed creation on decentralized platforms. The 2026 CHI paper “BONSAI” presents a system where users express feed intent in natural language, choose content sources, specify include/exclude prompts, and choose ranking priorities—while emphasizing procedural transparency and noting users can feel empowered but also face extra cognitive effort maintaining intentional feeds. ²²

The design takeaway for your idea is:

Natural language is a great “zero-friction start,” but it must be paired with: - visible translation into concrete controls (“this is what we will do”), - fast feedback loops (“preview shows the effect immediately”), - easy correction (“edit a rule, don’t rewrite the whole prompt”). ²³

This also aligns with broader work on making teachable feeds usable: transparency must make control actionable, and the feedback loop should be tight. ²³

UI/UX that non-technical users can actually use

Your success or failure is almost entirely determined by UX. The research is blunt: even when platforms provide powerful controls, adoption can be low because users don’t understand them, don’t trust them, or don’t see immediate benefits. ²⁴

A high-performing UI for “user-set algorithms” usually needs three layers, with progressive disclosure.

Layer one: “Goal-first onboarding” (what do you want to feel when you open the app?)

Instead of asking the user to design logic, ask them to choose an outcome: - “Catch up fast” - “See what’s blowing up right now” - “Avoid drama” - “Find new people” - “Only my communities” - “Deep reading mode”

This approach is consistent with research that users struggle to “formulate goals in a way that can be operationalized” in algorithmic settings; the UI must provide a language users can naturally use. ¹⁹

Layer two: “Controls embedded into the feed” (teach while browsing)

A key design objective identified in customizable feed research is **integration**: controls should fit the user’s existing workflow and not require mental context switching. ²⁰

This suggests a UI pattern like: - Tap a post’s “Why am I seeing this?” → shows which recipe rules caused it (source, topic match, boosted signal). - Tap “More like this” → adds a rule to the recipe (and shows the rule in plain language). - Tap “Too much of this” → adds a diversity cap or downranks a topic.

This is aligned with work on “teachable feed experiences” and usable controls that are “integrated directly into the feed.” ²⁰

Layer three: “Advanced builder” (your React Flow map), but redesigned for non-experts

The biggest risk with a node-based builder is that it *looks* powerful but causes failure for the exact users you care about (non-technical).

To make a visual builder usable, it needs to be: - **constrained** (only safe blocks), - **example-driven** (“this block will do X; here are 5 real posts it will filter”), - **immediately previewable** (every edit instantly updates a small preview pane), - **forgiving** (defaults and guardrails prevent “empty feed” or “infinite spam”). ²⁵

A strong compromise is: **the graph exists, but the user rarely sees it.**

Most users interact via recipe cards, sliders, and inline teaching; the graph becomes an “explain mode” and “expert mode.”

You can also borrow from the real-world lesson of Bluesky’s ecosystem: custom feeds were powerful but initially required developer familiarity; third-party clients like SkyFeed emerged specifically to make feed-building accessible using simplified constructs (regex/list-based rules). ²⁶

Trust, safety, and graceful failure without exposing code

Your constraints (no code, failure handling, no confusion on user switching) are exactly the right concerns. Research and policy both point to the same principle:

If you give users control but don’t give them understanding + recovery, the feature becomes either scary or ignored. ²⁷

Here is a clean “idea-level” safety model that stays within your requirements:

- **No user code, ever.** Users can only assemble *pre-approved primitives* (filters, boosts, caps, rank priorities). This mirrors proven “no-code feed” approaches. ¹²
- **Every recipe is validated and explainable.** If a recipe can’t be described in plain English, don’t allow it. This aligns with DSA expectations around intelligible explanation of main parameters and options. ²⁸
- **Hard constraints prevent catastrophic outcomes.** Example: “Never produce an empty feed; if rules filter everything, loosen the strictest filter and notify the user.” This kind of guardrail is implied by usability research highlighting cognitive load and the need for tight feedback loops. ²³
- **Always-on fallback:** if a recipe errors, times out, or yields invalid output, the system temporarily reverts to the default algorithm and shows a message like: “Your recipe couldn’t be applied right now; we’re showing Default while we retry.” (This is critical for real-time UX expectations.) The need for clear, immediate feedback is repeatedly emphasized in feed customization research. ²⁹
- **Recipes are versioned and reversible:** users can undo the last change (“Restore yesterday’s version”). This is a conceptual requirement stemming from the reality that users often experiment and develop folk theories; reversibility reduces fear. ²⁰

- **Clear “Current Recipe” indicator + instant switching:** prevent confusion when users switch accounts or switch recipe tabs by always showing the active recipe name and a one-line summary (“Trending Now: popularity high, freshness medium, sources = Following + Communities”). DSA and enforcement commentary emphasize that options must be directly accessible and not hidden behind friction. ³⁰

Real-time and scaling reality, translated into product constraints

You asked for a real-time project that can handle many user-specific algorithms. The conceptual insight here is:

Scalability is not only an infrastructure problem; it’s a “what do we allow users to express?” problem.

Bluesky’s feed generator model illustrates a scalable boundary: custom feeds return a ranked list of post identifiers; the client/app view hydrates them. ¹¹ But even in that ecosystem, latency can become real when feeds require expensive per-user fetching; academic work on a Bluesky feed (“Paper Skygest”) reports substantial latency for some users’ recommendations largely due to the need to fetch user follow graphs via API. ³¹

Your product implication: for “user algorithms,” you want to privilege recipes that can be expressed as: - supported filters over indexed attributes, - supported ranking signals you already compute, - limited lookbacks/time windows, - bounded “mixing” logic (e.g., “70% trending + 30% fresh”). ³²

This is also consistent with research showing that simple, controllable blends (e.g., popularity and age) are effective and computationally cheap. ¹⁷

Finally, there’s a real-world signal that “many feeds” can be operationally supported if the system is built around templates/configs: TechCrunch reports that Graze powered thousands of Bluesky feeds and focuses on templates, moderation logic, sort order, and social-graph inclusion. ³³ That doesn’t prove your specific scaling will be easy, but it supports the feasibility of an ecosystem where many distinct ranking configs exist simultaneously. ³⁴

Extensions that strengthen community building

Your idea becomes much more powerful (and more “community-building” than “algorithm nerd feature”) if you add one more concept:

Feed Curators + Shareable Recipes

Bluesky explicitly describes custom feeds as a marketplace-like ecosystem where people can choose feeds (friends-first, topic feeds, post-only feeds, etc.), and it points to third-party tools that help users create feeds and dashboards. ³⁵ Graze positions itself around letting people build custom feeds and foster topic communities, and TechCrunch describes customers using it to make niche social experiences with moderation constraints like “safe for work.” ³⁶

If you enable recipe sharing (copy, remix, subscribe), you get: - community “feed culture” (people rally around a recipe), - faster onboarding (new users pick a trusted recipe), - better safety outcomes (trusted curators maintain recipes), - less cognitive load (users don’t have to build from scratch). ³⁷

This also aligns with research systems like Gobo ³⁸, which explored user control and transparency to help users manage what is hidden, understand why they see content, and regain control from “invisible algorithms.” ³⁹

A key caution: giving users more control can also intensify filter-bubble behavior if the easiest recipes over-optimize for comfort. The empirical and policy debate on feed algorithms (especially around civic information) is active; the Science field experiments on social media algorithms during elections show why ranking choices matter and why “baseline feeds” and auditing frameworks are being discussed. ⁴⁰
This doesn’t argue against your idea—it argues for making “diversity,” “source transparency,” and “reset to baseline” first-class concepts in your recipe system. ⁴¹

1 About your For you timeline on X

https://help.x.com/en/using-x/x-timeline?utm_source=chatgpt.com

2 What are home feed recommendations? - Reddit Help

https://support.reddithelp.com/hc/en-us/articles/4402284777364-What-are-home-feed-recommendations?utm_source=chatgpt.com

3 Control Your Instagram Feed with Favorites and Following

https://about.instagram.com/blog/announcements/favorites-and-following?utm_source=chatgpt.com

4 13 14 16 18 24 User Control in Recommender Systems

https://web-ainf.aau.at/pub/jannach/files/Conference_EC_Web_2016.pdf?utm_source=chatgpt.com

5 9 10 35 Algorithmic Choice with Custom Feeds - Bluesky

https://bsky.social/about/blog/7-27-2023-custom-feeds?utm_source=chatgpt.com

6 26 Featured Community Project: Skyfeed

https://docs.bsky.app/blog/feature-skyfeed?utm_source=chatgpt.com

7 28 30 Article 27, the Digital Services Act (DSA)

https://www.eu-digital-services-act.com/Digital_Services_Act_Article_27.html?utm_source=chatgpt.com

8 Dutch court upholds ruling forcing Meta to offer chronological feeds

https://www.reuters.com/legal/litigation/dutch-court-upholds-ruling-forcing-meta-offer-chronological-feeds-2026-03-10/?utm_source=chatgpt.com

11 Feeds - AT Protocol Docs

https://atproto.com/guides/feeds?utm_source=chatgpt.com

12 Docs | Introduction

https://www.graze.social/docs/intro?utm_source=chatgpt.com

15 Letting Users Choose Recommender Algorithms

https://md.ekstrand.net/pubs/MultiRecs-Author.pdf?utm_source=chatgpt.com

17 32 files.grouplens.org

<https://files.grouplens.org/papers/harper-recsys2015.pdf>

19 20 21 25 27 29 38 **Designing Usable Controls for Customizable Social Media Feeds**

<https://arxiv.org/html/2509.19615v1>

22 23 37 **BONSAI: Intentional and Personalized Social Media Feeds**

<https://arxiv.org/html/2509.10776v2>

31 **Paper Skygest: Personalized Academic Recommendations ...**

https://arxiv.org/html/2601.04253v1?utm_source=chatgpt.com

33 34 **Bluesky feed builder Graze raises \$1M, rolls out ads**

https://techcrunch.com/2025/04/16/bluesky-feed-builder-graze-raises-1m-rolls-out-ads/?utm_source=chatgpt.com

36 **Feed Curators | Graze**

https://www.graze.social/feed-curators?utm_source=chatgpt.com

39 **Gobo: A System for Exploring User Control of Invisible ...**

https://www.cs.unc.edu/~gaikwad/assets/publications/cscw-gobo.pdf?utm_source=chatgpt.com

40 **How do social media feed algorithms affect attitudes and ...**

https://www.science.org/doi/10.1126/science.abp9364?utm_source=chatgpt.com

41 **A User-Driven Framework for Regulating and Auditing Social Media**

https://arxiv.org/abs/2304.10525?utm_source=chatgpt.com